

ASSIGNMENT – 3

URL Shortener with Redis Caching and MongoDB

AIM

To extend the URL Shortener from Experiment 2 by adding a Redis cache-aside layer on top of a MongoDB database. The goal is to reduce GET /{shortCode} response time: the first request is a cache miss (fetches from MongoDB and stores the result in Redis), and subsequent requests are cache hits served directly from Redis.

Team Details

S.No	Name	Register Number
1	Dinesh Kumar J	3122245001042
2	Gopinath S	3122245001048
3	Harish R	3122245001055
4	Srikrishna O S	3122245001312

CODE - pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.4</version>
    <relativePath/>
  </parent>

  <groupId>com.ssn</groupId>
  <artifactId>url-shortener</artifactId>
  <version>1.0.0</version>
  <name>url-shortener</name>

  <properties>
    <java.version>17</java.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
```

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-mongodb</artifactId>
</dependency>

<!-- new for exp3, this is what lets us use StringRedisTemplate in the controller -->
<!-- spring boot autoconfigures the redis connection just from this + the host/port in application.properties -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
</project>
```

CODE - UrlShortenerController.java

```
package com.ssn.urlshortener;

import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.util.Base64;
import java.util.Optional;

@RestController //class handles http requests
public class UrlShortenerController { //controller

    private final UrlMappingRepository repository; //stores repository object which is used to communicate with the db
    private final StringRedisTemplate redis; //talks to memurai/redis, key = shortcode value = longurl

    public UrlShortenerController(UrlMappingRepository repository, StringRedisTemplate redis) {
        this.repository = repository;
        this.redis = redis;
    }

    @PostMapping("/shorten") //first api
    public ResponseEntity<ShortenResponse> shorten(@RequestBody ShortenRequest request) {

        String longUrl = request.getLongUrl();
```

```
String shortCode = generateUniqueShortCode(longUrl);

UrlMapping mapping = new UrlMapping(shortCode, longUrl);
repository.save(mapping);

String shortUrl = "http://localhost:8080/" + shortCode;
ShortenResponse response = new ShortenResponse(shortUrl);

return ResponseEntity.status(HttpStatus.CREATED).body(response);
}

@GetMapping("/{shortCode}")
public ResponseEntity<Void> redirect(@PathVariable String shortCode) {

    String longUrl = redis.opsForValue().get(shortCode);

    if (longUrl != null) {
        return ResponseEntity.status(HttpStatus.FOUND)
            .header(HttpHeaders.LOCATION, longUrl)
            .build();
    }

    Optional<UrlMapping> result = repository.findByShortCode(shortCode);

    if (result.isEmpty()) {
        return ResponseEntity.notFound().build();
    }

    longUrl = result.get().getLongUrl();

    redis.opsForValue().set(shortCode, longUrl);

    return ResponseEntity.status(HttpStatus.FOUND)
        .header(HttpHeaders.LOCATION, longUrl)
        .build();
}

private String generateUniqueShortCode(String longUrl) {

    String hash = sha256Base64(longUrl);
    int length = 6;
    String code = hash.substring(0, length); // to get only first few characters

    while (repository.findByShortCode(code).isPresent() && length < 8) {
        length++;
        code = hash.substring(0, length);
    }

    return code;
}

private String sha256Base64(String input) {
    try {
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        byte[] hashBytes = digest.digest(input.getBytes(StandardCharsets.UTF_8));
```

```
        return Base64.getEncoder().withoutPadding().encodeToString(hashBytes);

    } catch (Exception e) {
        throw new RuntimeException("Error generating hash", e);
    }
}
}
```

OUTPUT

```
POST /shorten
curl -i -X POST http://localhost:8080/shorten -H "Content-Type: application/json" -d @body.json

HTTP/1.1 201 Created
Content-Type: application/json
{"shortUrl":"http://localhost:8080/Ayirwc"}

GET /Ayirwc (cache miss)
curl -i -w " time_total: %{time_total}" http://localhost:8080/Ayirwc

HTTP/1.1 302 Found Location:
https://www.google.com/search?q=system+design
time_total: 0.295927 s

GET /Ayirwc (cache hit)
curl -i -w " time_total: %{time_total}" http://localhost:8080/Ayirwc

HTTP/1.1 302 Found Location:
https://www.google.com/search?q=system+design
time_total: 0.004459 s
```

SCREENSHOTS

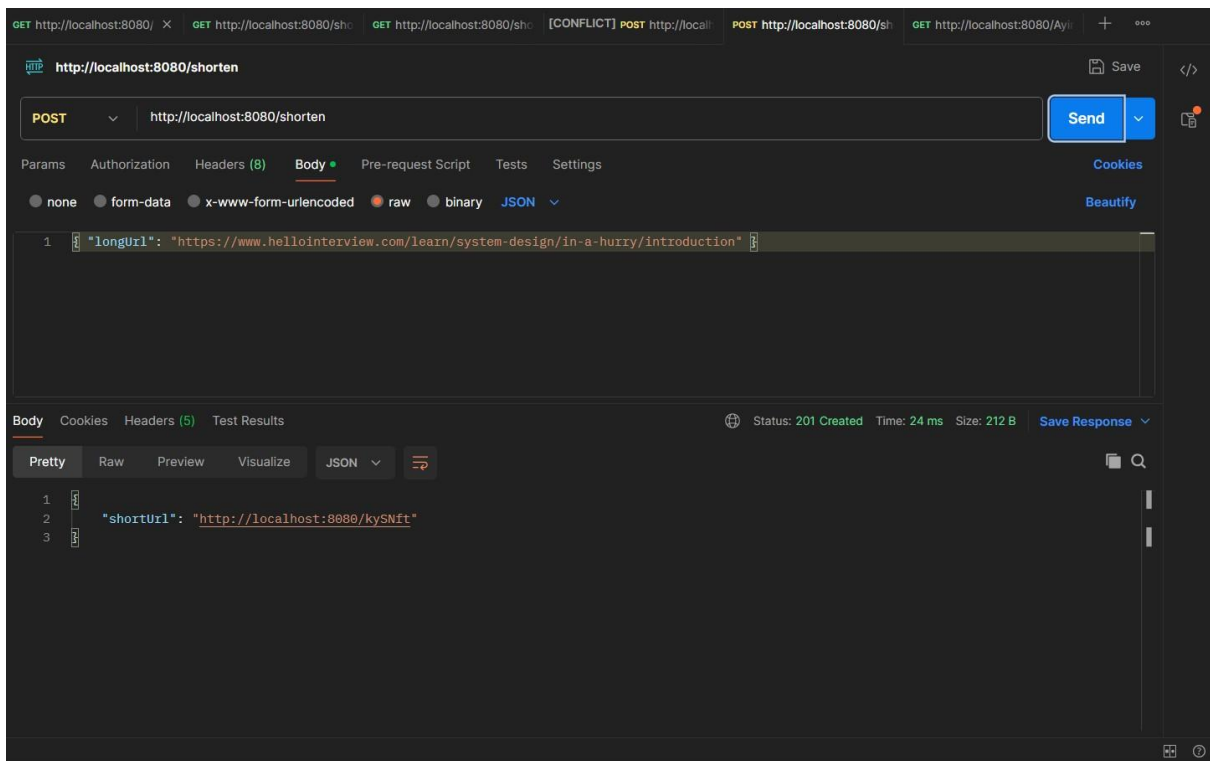


Fig 1 - POST /shorten request and response in Postman

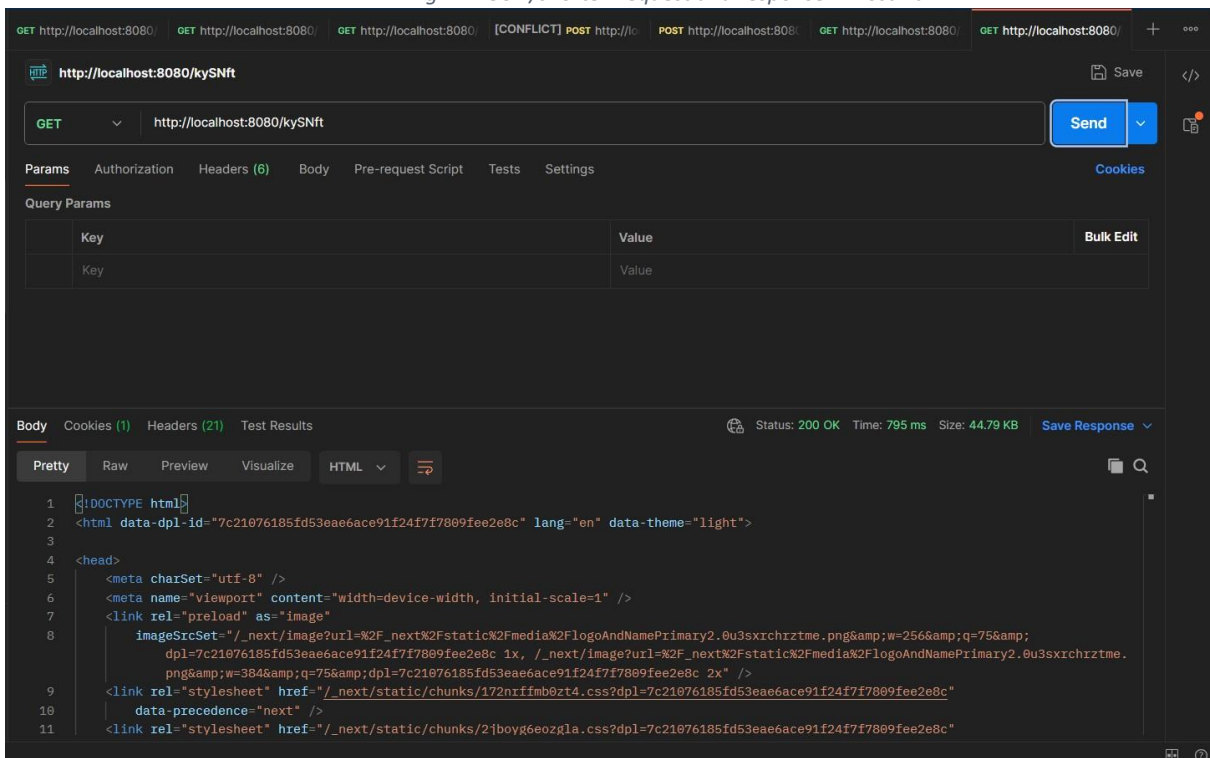


Fig 2 - GET /Ayirwc cache miss: MongoDB lookup took 295.93 ms

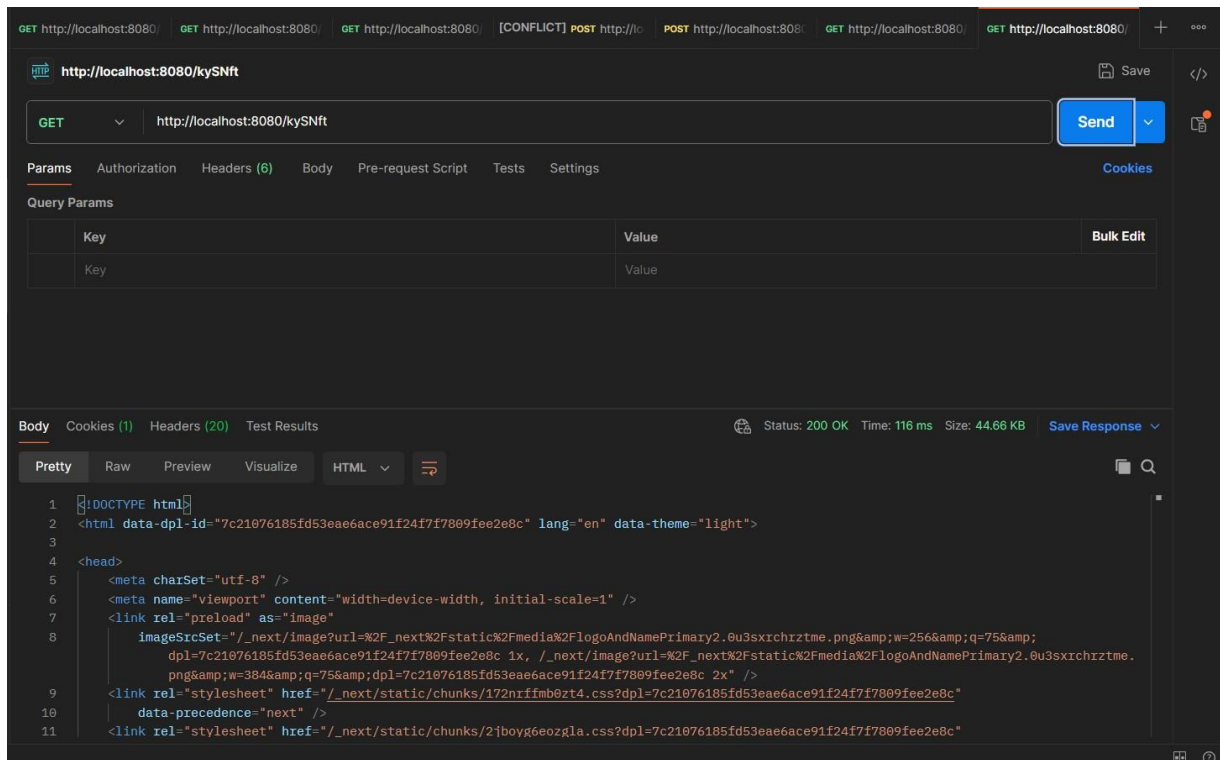


Fig 3 - GET /Ayirwc cache hit: Redis lookup took 4.46 ms

PERFORMANCE METRICS

The cache performance was measured by clearing Redis and sending two consecutive GET /{shortCode} requests. The first request is a cache miss because the long URL is not in Redis, so the application fetches it from MongoDB. The second request is a cache hit because the long URL was stored in Redis on the first request.

Endpoint	Method	Status	Time (ms)	Notes
POST /shorten	POST	201 Created	-	Returns the generated short URL
GET /Ayirwc	GET	302 Found	295.93	Cache miss: MongoDB lookup
GET /Ayirwc	GET	302 Found	4.46	Cache hit: Redis lookup

TASKS PERFORMED

Task	Description	Result / Output
Set up dependencies	Replaced JPA/H2 with spring-boot-starter-data-mongodb and kept spring-boot-starter-data-redis	MongoDB + Redis dependencies in pom.xml
Configure MongoDB	Updated application.properties to use mongodb://localhost:27017/urlshortener_exp3	App connects to MongoDB
Convert domain + repository	Switched UrlMapping to @Document and UrlMappingRepository to MongoRepository<String>	Documents stored in url_mapping collection

Task	Description	Result / Output
Implement POST /shorten	Creates short code, stores mapping in MongoDB	201 Created with shortUrl
Implement cache-aside GET	Checks Redis first; on miss reads MongoDB and writes to Redis	302 Found on both miss and hit
Test cache behavior	Flushed Redis, POSTed a long URL, then GET the same short code twice	Cache miss = 295.93 ms, cache hit = 4.46 ms
Generate screenshots	Captured POST, cache-miss GET, and cache-hit GET responses	Three Postman screenshots included

LEARNING OUTCOMES

- Understood how to add a Redis cache-aside pattern to an existing Spring Boot service.
- Observed the real performance difference between a cache miss (MongoDB lookup) and a cache hit (Redis lookup).
- Practiced switching a Spring Boot app from JPA/H2 to Spring Data MongoDB.
- Reinforced the use of HTTP 302 redirects for short URL resolution.
- Learned that cache eviction policies and memory caps are configured on the Redis server, not in the application.